

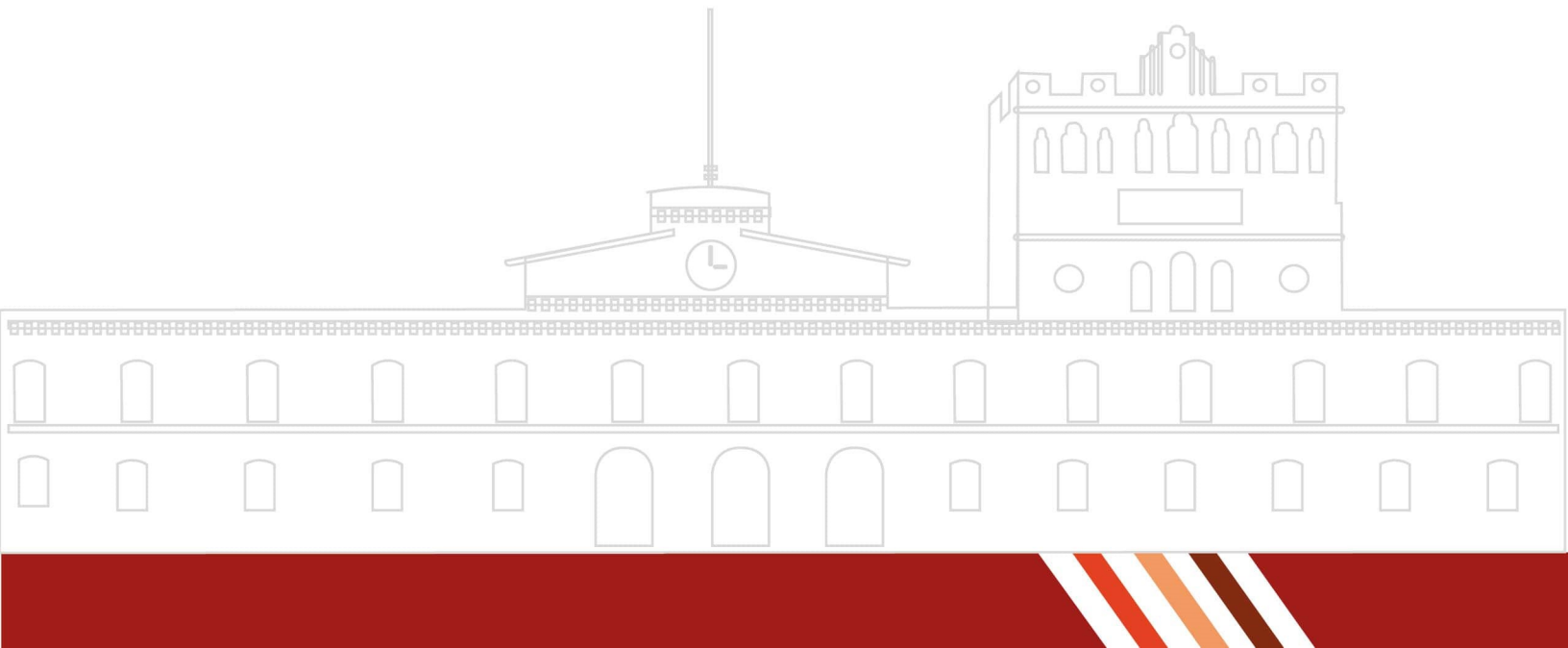
REPORTE DE PRÁCTICA 7

NOMBRE DE LA PRÁCTICA: Analizador léxico-sintáctico

ALUMNOS:

- Cortés Santos María Fernanda
- Jácome Cayetano José David
- Palma Velazquez Ariadna Betsabé

Dr. Eduardo Cornejo-Velázquez



Introducción

El proceso de construcción de un compilador o intérprete consta de varias fases fundamentales que permiten a la computadora comprender un lenguaje de programación. Las dos primeras y más críticas son el análisis léxico y el análisis sintáctico. En el desarrollo de la presente práctica, se expone el diseño e implementación de un analizador léxico-sintáctico haciendo uso de las herramientas generadoras Lex/Flex y Yacc/Bison. El objetivo principal de este proyecto es construir un programa capaz de leer código fuente, tokenizar sus componentes y validar que su estructura gramatical responda a las reglas definidas.

Marco teórico

Fases de un Compilador

El proceso de traducción de un lenguaje de alto nivel a un lenguaje comprensible por la máquina se divide en varias fases analíticas y sintéticas. Las dos primeras, y fundamentales para el procesamiento del código fuente, son el análisis léxico y el análisis sintáctico. Estas etapas trabajan en conjunto para garantizar que el programa de entrada esté correctamente escrito tanto a nivel de vocabulario como de estructura gramatical [1].

Análisis Léxico y Tokens

El análisis léxico (o escaneo) es la fase inicial del compilador. Su función principal es leer el flujo de caracteres del código fuente, de izquierda a derecha, y agruparlos en secuencias lógicas con significado, denominadas tokens [2]. Un token representa categorías léxicas predefinidas, tales como identificadores de variables, palabras reservadas, constantes numéricas (enteros, flotantes, octales, etc.) u operadores. Durante este proceso, el analizador también se encarga de descartar elementos no ejecutables, como los espacios en blanco, tabulaciones y comentarios [3].

Expresiones Regulares y Autómatas

Para que el analizador léxico pueda identificar y clasificar los patrones que conforman los tokens, se fundamenta en la teoría de lenguajes regulares. Las expresiones regulares son notaciones algebraicas que describen formalmente el formato de las cadenas permitidas. Computacionalmente, estas expresiones se evalúan mediante autómatas finitos (deterministas o no deterministas), los cuales dictan las transiciones de estado necesarias para reconocer el vocabulario del lenguaje [2].

Análisis Sintáctico y Gramáticas Libres de Contexto

El análisis sintáctico (o parsing) toma la secuencia de tokens generada por el analizador léxico y verifica si dicha secuencia forma expresiones y sentencias válidas [4]. Esta validación se rige por las Gramáticas Libres de Contexto (GLC), las cuales establecen reglas jerárquicas y de derivación. Gracias a estas reglas, el analizador puede resolver ambigüedades, como respetar la precedencia de los operadores matemáticos (por ejemplo, que la multiplicación se evalúe antes que la suma) y el uso correcto de paréntesis para agrupar expresiones [1].

Herramientas Generadoras: Lex/Flex y Yacc/Bison

Dado que la construcción manual de analizadores es un proceso extenso y propenso a errores, en la ingeniería de software se emplean herramientas generadoras. Lex (y su versión libre, Flex) es un generador de analizadores léxicos que produce código en C a partir de reglas basadas en expresiones regulares [5]. Por su parte, Yacc (y su contraparte Bison) es un generador de analizadores sintácticos ascendentes (usualmente de tipo LALR) que procesa una gramática libre de contexto [5]. Ambas herramientas están diseñadas para

interactuar de forma fluida: el analizador sintáctico (Yacc) invoca iterativamente la función `yylex()` proporcionada por Lex para solicitar el siguiente token del flujo de entrada y así continuar con la validación de las sentencias [4].

Herramientas empleadas

1. Lex (o Flex): Empleado como el generador del analizador léxico.
2. Yacc (o Bison): Utilizado como el generador del analizador sintáctico.
3. Lenguaje C: lenguaje de programación base.
4. Archivos de texto plano (.txt): Se utilizaron como la interfaz de entrada y salida de datos.

Desarrollo

El archivo léxico fue programado para actuar como un escáner basado en expresiones regulares. Se incluyó la librería `y.tab.h` (generada por Yacc) para sincronizar los identificadores de los tokens entre ambos archivos. Entre las implementaciones más destacadas se encuentran:

- Reconocimiento numérico avanzado: Se utilizaron expresiones regulares específicas para identificar distintas bases numéricas. Además, se hizo uso de funciones como `strtol()` y `atof()` para convertir las cadenas de texto a sus valores numéricos reales y almacenarlos en la variable semántica `yyval`.
- Clasificación de palabras reservadas: Se definió un extenso bloque de reglas que reconocen la mayoría de las palabras clave de lenguajes como C/C++ (`auto`, `break`, `class`, `int`, etc.).
- Se configuraron reglas para detectar y aislar comentarios de una (`//`) y múltiples líneas (`/* ... */`), así como para ignorar saltos de línea y tabulaciones, evitando que esta información irrelevante llegue al analizador sintáctico.

El archivo sintáctico define la estructura gramatical del lenguaje a través de una Gramática Libre de Contexto (GLC) y procesa los tokens recibidos.

- Se implementó la directiva `%union` para permitir que los tokens transporten diferentes tipos de datos (enteros, flotantes, cadenas de caracteres).
- Se utilizó la directiva `%left` para los operadores y comparadores matemáticos. Crucial para resolver ambigüedades, asegurando que operaciones como la multiplicación se evalúen antes que la suma, replicando el comportamiento algebraico estándar.
- Se diseñó una estructura jerárquica donde el axioma principal (`input`) puede derivar en múltiples líneas (`linea`), las cuales a su vez contienen expresiones (`expresion`) compuestas por valores (`valor`). El sistema cuenta con bloques de código en C incrustados en las reglas para imprimir en consola (y en el archivo de salida) la clasificación exacta de cada componente detectado.

En la función `main()`, se implementó una redirección de los flujos estándar de entrada y salida. En lugar de requerir que el usuario escriba comandos en la consola, el programa utiliza `fopen` para leer el código fuente desde `entrada.txt` y `freopen` para volcar todos los resultados del análisis (`printf`) directamente en un archivo `salida.txt`. Esto automatiza el proceso de prueba y permite evaluar bloques grandes de código de forma simultánea.

Orden de Compilación

Para la correcta construcción y ejecución del analizador léxico-sintáctico, se tuvo que seguir el siguiente orden de compilación para garantizar que las dependencias entre archivos se generen adecuadamente:

1. Generación preliminar desde la interfaz gráfica
 - Archivo léxico (.l): Abrir el archivo en el editor y seleccionar la herramienta "1" para procesar las expresiones regulares con Flex.
 - Archivo sintáctico (.y): Abrir el archivo y seleccionar consecutivamente las herramientas "3" y "4" para procesar la gramática base con Bison/Yacc.
2. Compilación y enlazado mediante la terminal (PowerShell). Abrir la consola de comandos directamente en el directorio del proyecto y ejecutar la siguiente secuencia:
 - `bison -d Practica7.y`: Compilar la gramática libre de contexto. La bandera -d es fundamental, ya que genera el archivo de cabecera con las definiciones de los tokens que el analizador léxico requiere para funcionar.
 - `flex Practica7.l`: Compilar las reglas léxicas. Flex toma las definiciones generadas en el paso anterior y crea el archivo fuente en C (`lex.yy.c`).
 - `gcc y.tab.c lex.yy.c -o Practica7.exe`: Enlazar y compilar mediante GCC los archivos C generados (léxico y sintáctico) para crear un único programa ejecutable nombrado `Practica7.exe`.
 - `./Practica7.exe`: Ejecutar el programa final. El analizador se iniciará automáticamente, procesando el código fuente de `entrada.txt` para generar los resultados de la evaluación.

Capacidades del Sistema

- Es capaz de procesar e identificar la sintaxis básica y los tipos de datos de lenguajes de alto nivel orientados a objetos y estructurados.
- Valida correctamente la estructura de operaciones matemáticas complejas que incluyen agrupaciones por paréntesis y múltiples operadores.
- Manejo de archivos de texto plano, puede escanear y evaluar cientos de líneas de código de forma ininterrumpida, generando un reporte final automatizado.
- Mediante la función `yyerror`, el sistema es capaz de detectar cuándo una cadena de tokens no cumple con las reglas gramaticales y emitir el aviso de "EXPRESION INVALIDA".
- El sistema permite la sintaxis de expresiones independientemente de la lógica semántica.

Posibles Áreas de Mejora

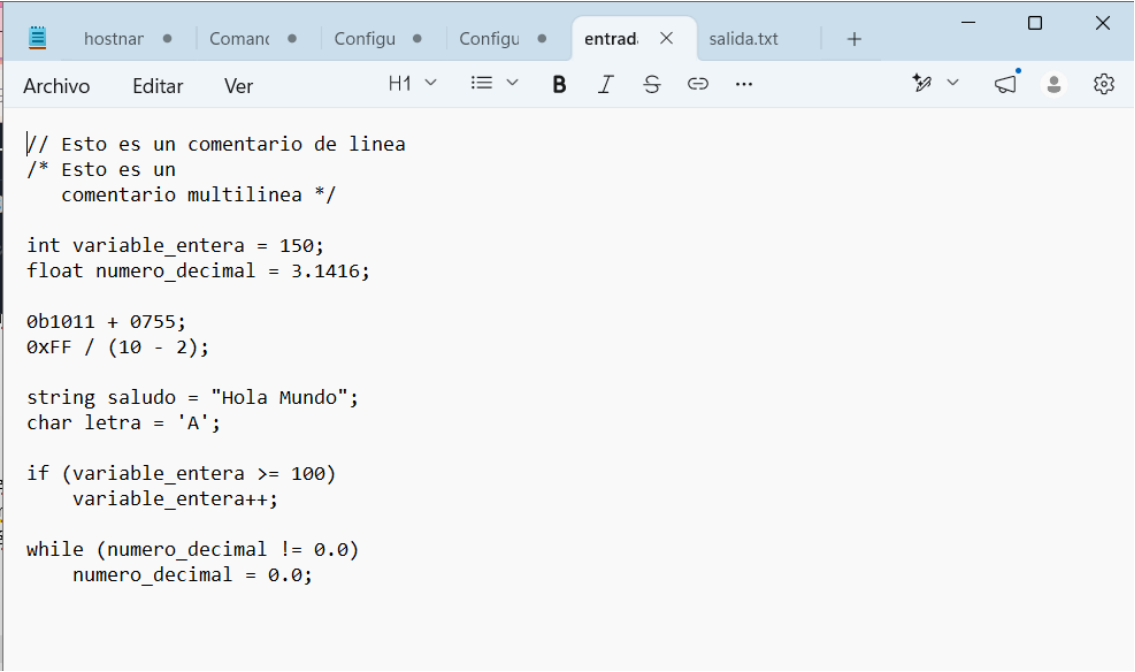
- Actualmente, cuando ocurre un error sintáctico, la función `yyerror` emite un mensaje genérico. Se podría implementar un contador de líneas para indicar en qué línea y columna ocurrió el fallo.

Código

- Archivo.l https://github.com/MaFecs/Actividades_Automatas_y_Compiladores/blob/ad0da4c83aebc363166c416Analizador_Sintactico/Practica7.1
- Archivo.y https://github.com/MaFecs/Actividades_Automatas_y_Compiladores/blob/ad0da4c83aebc363166c416Analizador_Sintactico/Practica7.y

Resultados

Archivo de entrada

A screenshot of a code editor window. The title bar shows tabs for 'hostnar', 'Comand', 'Configu', 'Configu', 'entrad', and 'salida.txt'. The menu bar includes 'Archivo', 'Editar', and 'Ver'. The toolbar shows 'H1', a list icon, 'B' (bold), 'I' (italic), 'S' (underline), a link icon, and a search icon. The code is written in C++ and includes comments, variable declarations, arithmetic expressions, string and character declarations, and control flow statements.

```
// Esto es un comentario de linea
/* Esto es un
   comentario multilinea */

int variable_entera = 150;
float numero_decimal = 3.1416;

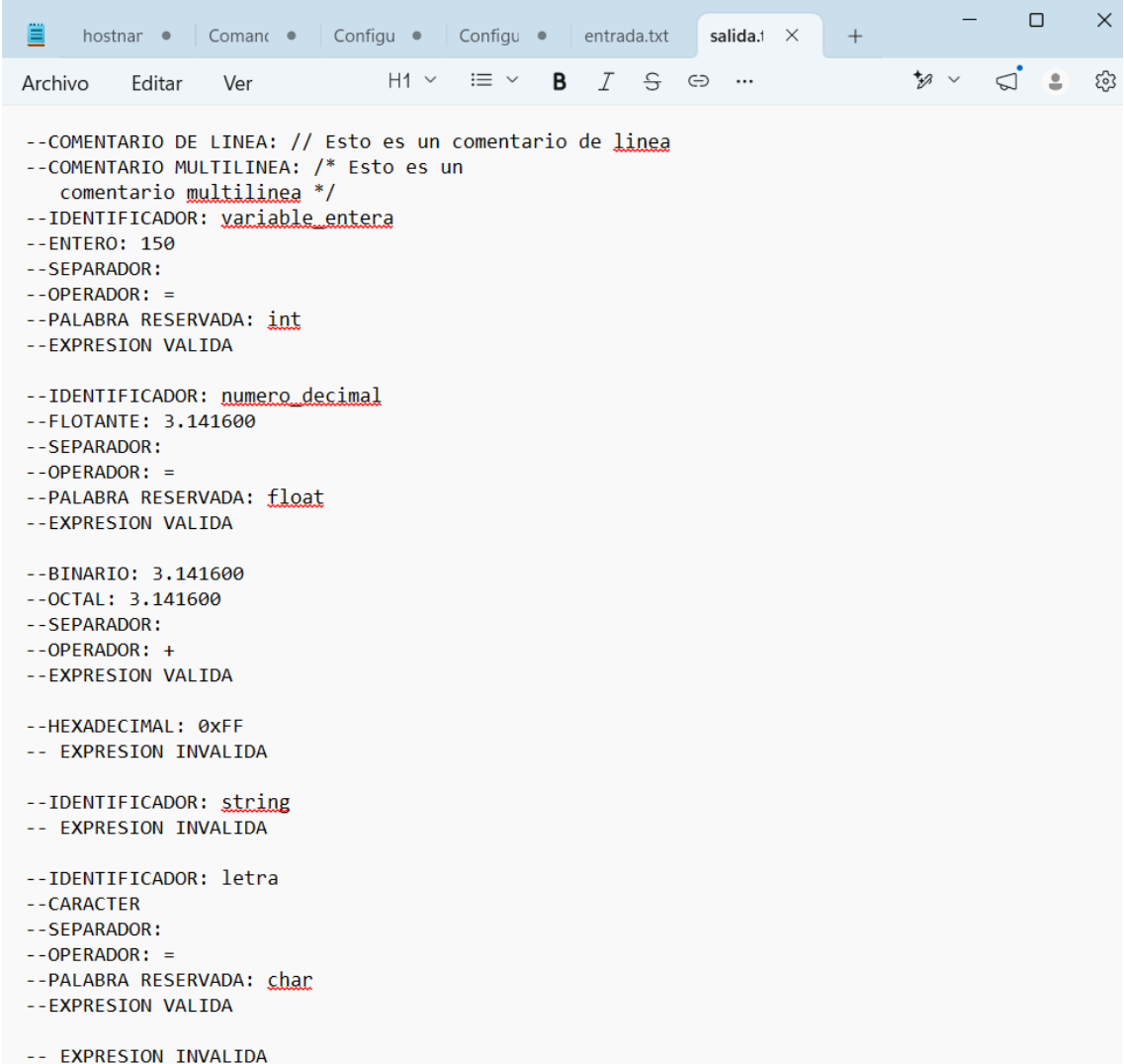
0b1011 + 0755;
0xFF / (10 - 2);

string saludo = "Hola Mundo";
char letra = 'A';

if (variable_entera >= 100)
    variable_entera++;

while (numero_decimal != 0.0)
    numero_decimal = 0.0;
```

Archivo de salida



```
--COMENTARIO DE LINEA: // Esto es un comentario de linea
--COMENTARIO MULTILINEA: /* Esto es un
    comentario multilinea */
--IDENTIFICADOR: variable_entera
--ENTERO: 150
--SEPARADOR:
--OPERADOR: =
--PALABRA RESERVADA: int
--EXPRESION VALIDA

--IDENTIFICADOR: numero_decimal
--FLOTANTE: 3.141600
--SEPARADOR:
--OPERADOR: =
--PALABRA RESERVADA: float
--EXPRESION VALIDA

--BINARIO: 3.141600
--OCTAL: 3.141600
--SEPARADOR:
--OPERADOR: +
--EXPRESION VALIDA

--HEXADECIMAL: 0xFF
-- EXPRESION INVALIDA

--IDENTIFICADOR: string
-- EXPRESION INVALIDA

--IDENTIFICADOR: letra
--CARACTER
--SEPARADOR:
--OPERADOR: =
--PALABRA RESERVADA: char
--EXPRESION VALIDA

-- EXPRESION INVALIDA
```

Conclusiones

A lo largo de esta práctica, pudimos aterrizar conceptos teóricos de la materia, como el comportamiento de los autómatas y las gramáticas libres de contexto, en un programa completamente funcional. Trabajar en conjunto con Lex y Yacc nos ayudó a entender cómo se comunican las dos primeras fases de un compilador sin la necesidad de programar un escáner o un parser desde cero, ya que estas herramientas facilitan muchísimo el manejo de expresiones regulares y reglas gramaticales complejas.

Al final, logramos construir un analizador léxico-sintáctico capaz de leer un archivo fuente y validar correctamente si su estructura es válida o no. El desarrollo de este proyecto nos deja ver la gran importancia de tener un "filtro" estricto en el código, dándonos una base sólida para comprender cómo la computadora logra interpretar y traducir los lenguajes de programación.

Referencias Bibliográficas

- 1 Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). Compiladores: principios, técnicas y herramientas (2a. ed.). Pearson Educación.
- 2 Hopcroft, J. E., Motwani, R., y Ullman, J. D. (2007). Introducción a la teoría de autómatas, lenguajes y computación (3a. ed.). Pearson Educación.
- 3 Louden, K. C. (2004). Construcción de compiladores: principios y práctica. Thomson.
- 4 Levine, J. R., Mason, T., y Brown, D. (1992). Lex y Yacc (2nd ed.). O'Reilly Media.
- 5 Grune, D., Bal, H. E., Jacobs, C. J., y Langendoen, K. G. (2012). Modern Compiler Design (2nd ed.). Springer.